

Graph Neural Networks: Model III

Jiaxuan You

Assistant Professor at UIUC CDS



CS512: Data Mining Principles, 2025 Fall

<https://ulab-uiuc.github.io/CS512/>

Reminder: Writing and Proposal Task

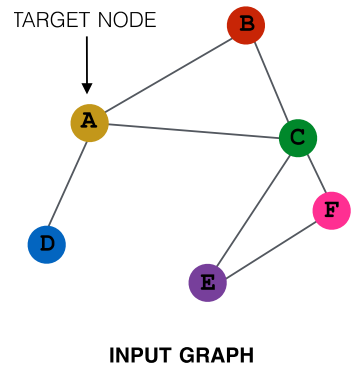
- **Reminder:** Writing task 2 released, due **Sept 24 11:59pm**
 - [Template](#) has been released on Canvas
- **Reminder:** Nominate papers after the discussion, due **Sept 24 11:59pm**
 - Fill in the details of recommended papers on Google Form [[Link](#)]
 - Count **2%** toward the “Discussion” part of grade. Should take <15 min to finish
- **Find teammates for course project**
 - **3 people per group** by default, sign up on Google Sheets [[link](#)]
- **Reminder:** Idea proposal task 2 released, due **Sept 30 11:59pm**
 - **Instructions** has been released on Canvas
 - **1-1.5 page of idea** in ICLR format. No need to execute in this course
- Coding Assignment 1 will be release tonight, due **Oct 3 11:59 PM, CT**
 - Submit your code and written answers downloaded from Colab to Canvas

Graph Neural Networks: Model II

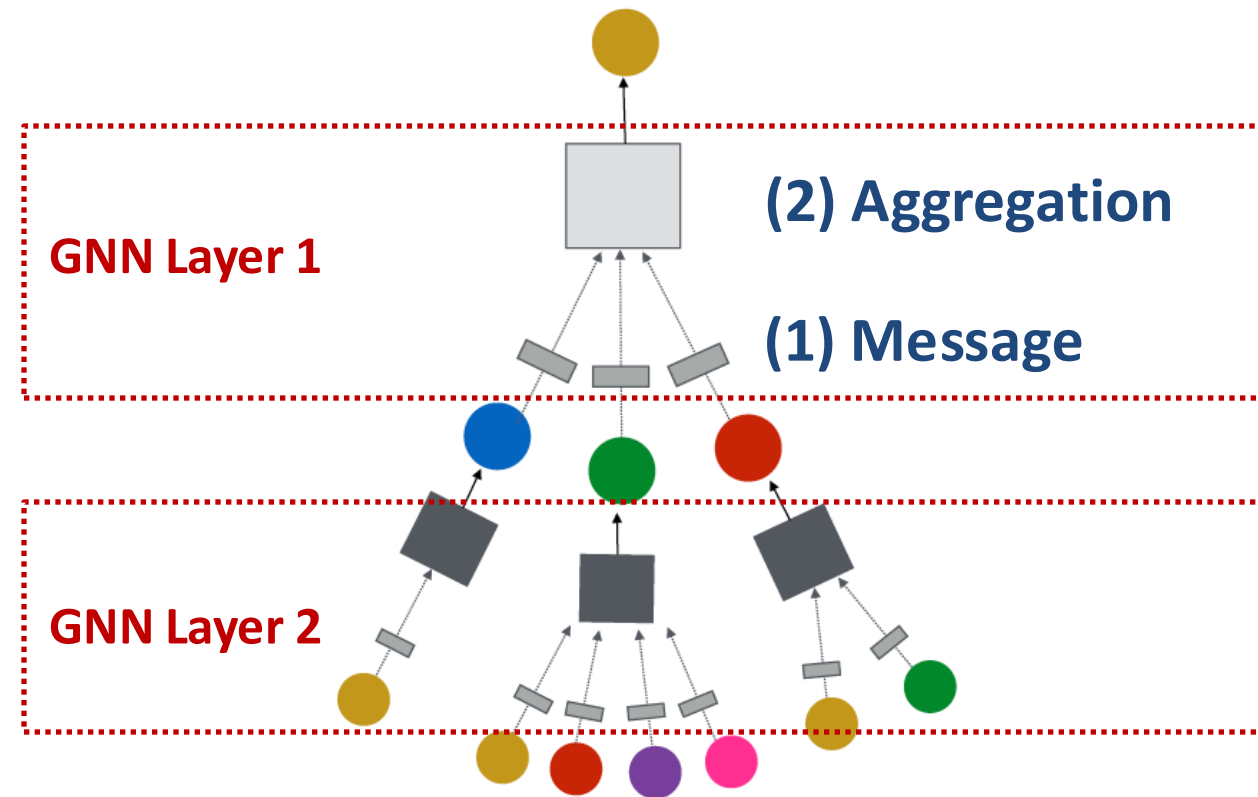
GNN Augmentation and Training

Recap: A General GNN Framework

(5) Learning objective



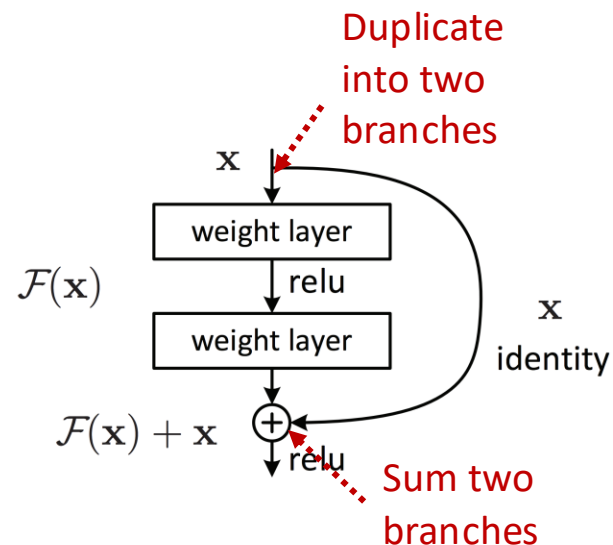
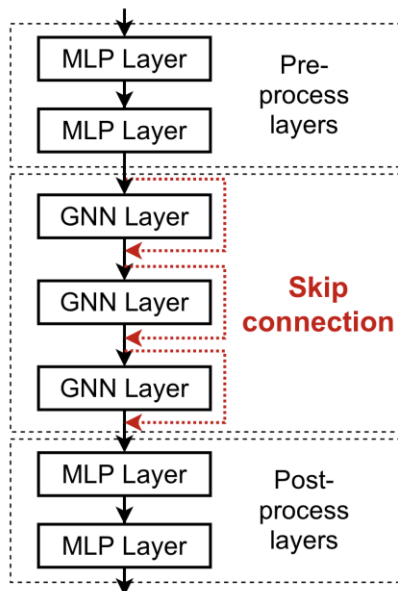
(3) Layer connectivity



(4) Graph augmentation

Recap: GNN Layer Connectivity

- What if my problem still requires many GNN layers?
- Lesson 2: Add skip connections in GNNs
 - **Observation from over-smoothing:** Node embeddings in earlier GNN layers can sometimes better differentiate nodes
 - **Solution:** We can increase the impact of earlier layers on the final node embeddings, **by adding shortcuts in GNN**



Idea of skip connections:

Before adding shortcuts:

$$F(x)$$

After adding shortcuts:

$$F(x) + x$$

Recap: Graph Manipulation

- **Graph Feature manipulation**

- The input graph **lacks features** → **feature augmentation**

- **Graph Structure manipulation**

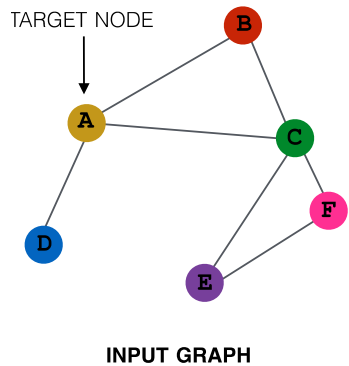
- The graph is **too sparse** → **Add virtual nodes / edges**
- The graph is **too dense** → **Sample neighbors when doing message passing**
- The graph is **too large** → **Sample subgraphs to compute embeddings**
 - Will cover later in lecture: Scaling up GNNs

Graph Neural Networks: Model II

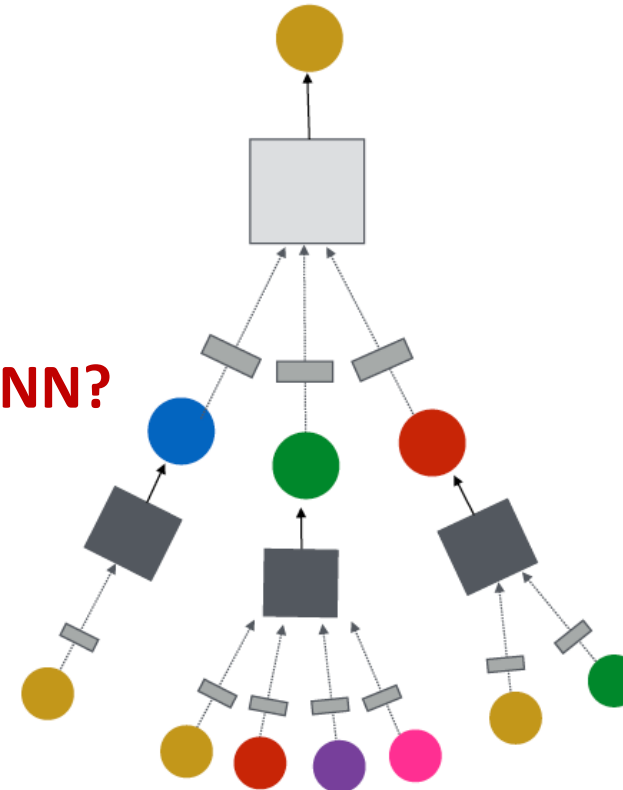
Prediction with GNNs

A General GNN Framework (4)

(5) Learning objective

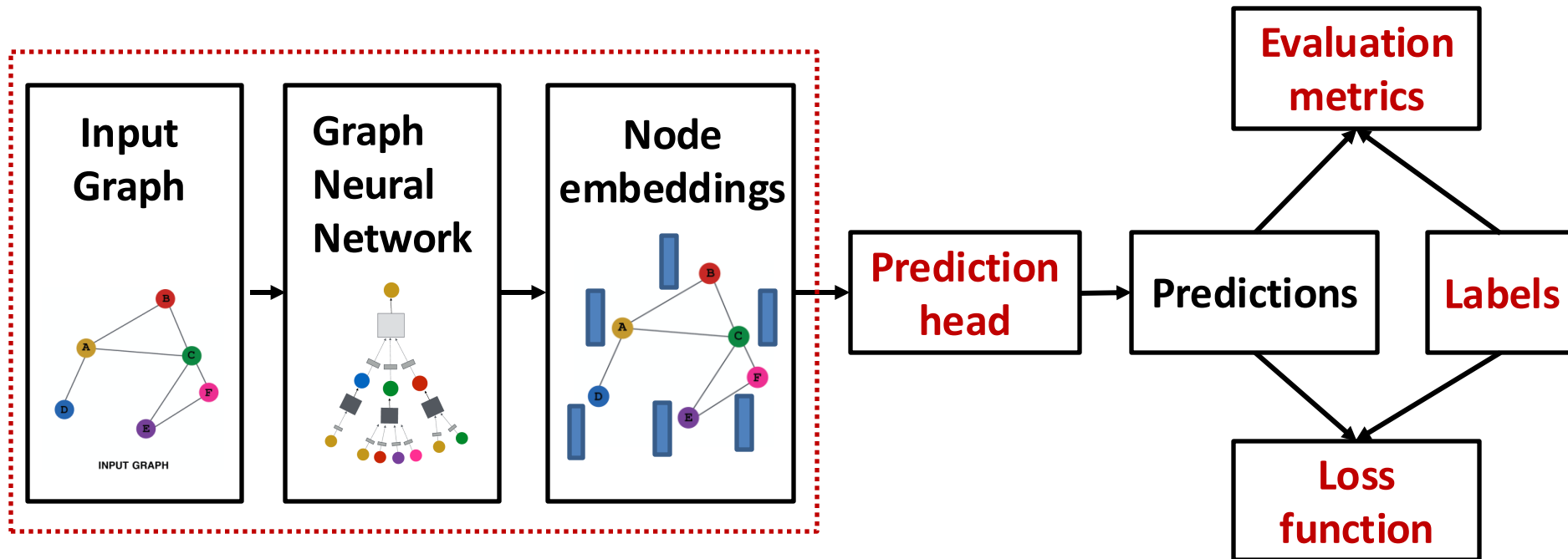


Next: How do we train a GNN?



GNN Training Pipeline

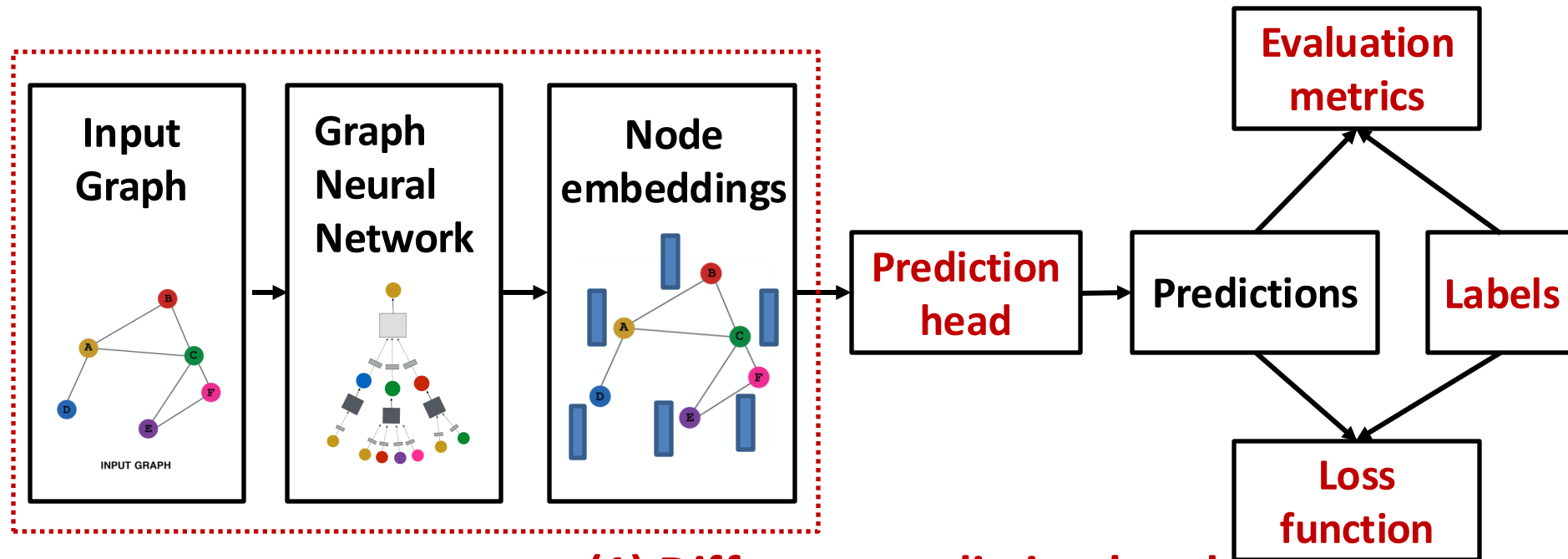
So far what we have covered



Output of a GNN: set of node embeddings

$$\{\mathbf{h}_v^{(L)}, \forall v \in G\}$$

GNN Training Pipeline (1)

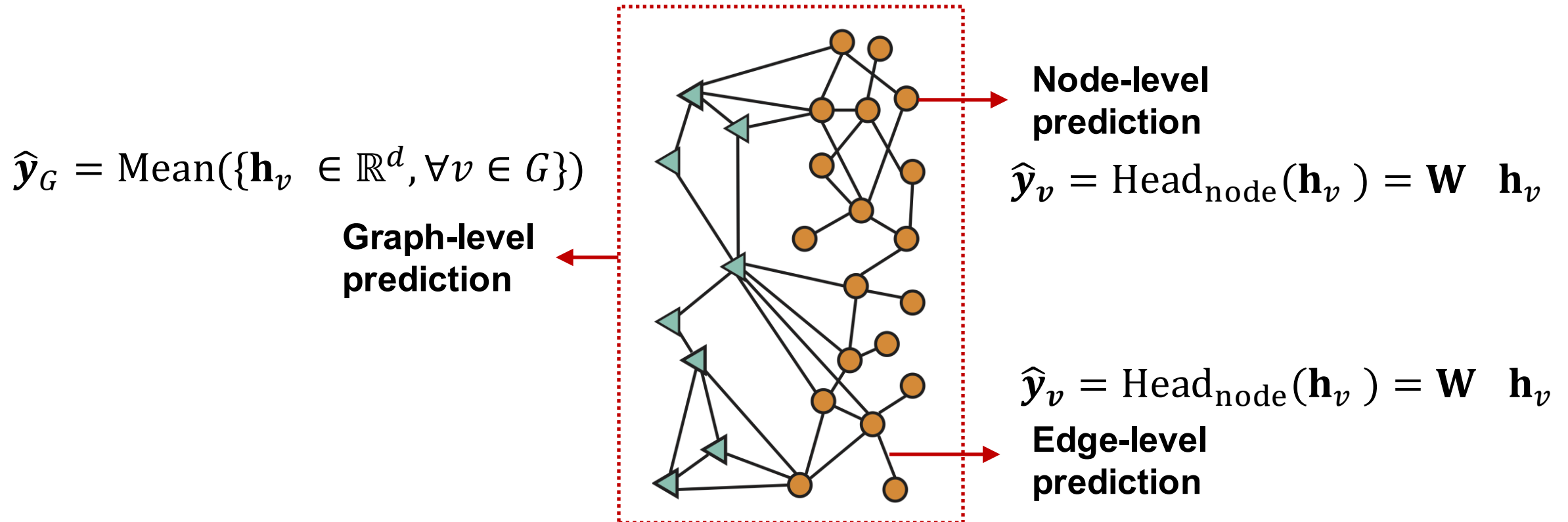


(1) Different prediction heads:

- Node-level tasks
- Edge-level tasks
- Graph-level tasks

Recap: Graph Learning Prediction Heads

- **Idea:** Different task levels require different prediction heads
- **Prediction head:** map **node embeddings** to the **predictions of interest**



Recap: Prediction Heads: Graph-level

- Options for $\text{Head}_{\text{graph}}(\{\mathbf{h}_v \in \mathbb{R}^d, \forall v \in G\})$

- (1) Global mean pooling**

$$\hat{\mathbf{y}}_G = W \text{Mean}(\{\mathbf{h}_v \in \mathbb{R}^d, \forall v \in G\})$$

- (2) Global max pooling**

$$\hat{\mathbf{y}}_G = W \text{Max}(\{\mathbf{h}_v \in \mathbb{R}^d, \forall v \in G\})$$

- (3) Global sum pooling**

$$\hat{\mathbf{y}}_G = W \text{Sum}(\{\mathbf{h}_v \in \mathbb{R}^d, \forall v \in G\})$$

- (4) Global attention pooling**

$$\hat{\mathbf{y}}_G = W \text{Sum}(\{\alpha_v \mathbf{h}_v \in \mathbb{R}^d, \forall v \in G\}), \alpha_v = \frac{\exp(W h_v + b)}{\sum_{u \in G} \exp(W h_u + b)}$$

- These options work great for small graphs
- Can we do better for large graphs?**

Issue of Global Pooling

- **Issue:** Global pooling over a (large) graph will lose information
- **Toy example:** we use 1-dim node embeddings
 - Node embeddings for G_1 : $\{-1, -2, 0, 1, 2\}$
 - Node embeddings for G_2 : $\{-10, -20, 0, 10, 20\}$
 - Clearly G_1 and G_2 have very different node embeddings → Their structures should be different
- **If we do global sum pooling:**
 - Prediction for G_1 : $\hat{y}_G = \text{Sum}(\{-1, -2, 0, 1, 2\}) = 0$
 - Prediction for G_2 : $\hat{y}_G = \text{Sum}(\{-10, -20, 0, 10, 20\}) = 0$
 - We cannot differentiate G_1 and G_2 !

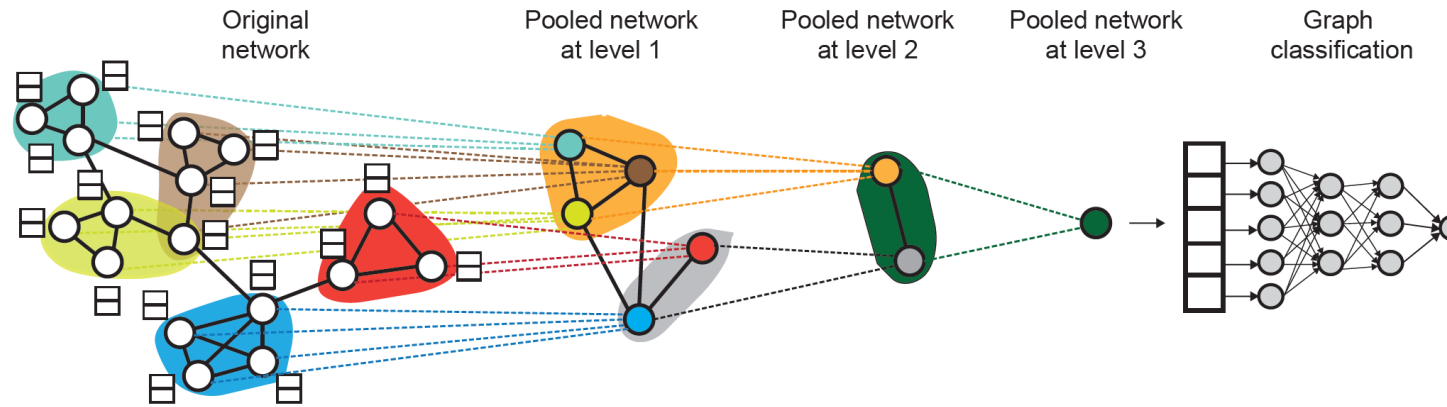
Hierarchical Global Pooling

- **A solution:** Let's aggregate all the node embeddings **hierarchically**
 - **Toy example:** We will aggregate via $\text{ReLU}(\text{Sum}(\cdot))$
 - We first **separately** aggregate the first 2 nodes and last 3 nodes
 - Then we aggregate again to make the final prediction
 - G_1 node embeddings: $\{-1, -2, 0, 1, 2\}$
 - **Round 1:** $\hat{y}_a = \text{ReLU}(\text{Sum}(\{-1, -2\})) = 0$, $\hat{y}_b = \text{ReLU}(\text{Sum}(\{0, 1, 2\})) = 3$
 - **Round 2:** $\hat{y}_G = \text{ReLU}(\text{Sum}(\{y_a, y_b\})) = 3$
 - G_2 node embeddings: $\{-10, -20, 0, 10, 20\}$
 - **Round 1:** $\hat{y}_a = \text{ReLU}(\text{Sum}(\{-10, -20\})) = 0$, $\hat{y}_b = \text{ReLU}(\text{Sum}(\{0, 10, 20\})) = 30$
 - **Round 2:** $\hat{y}_G = \text{ReLU}(\text{Sum}(\{y_a, y_b\})) = 30$

Now we can
differentiate
 G_1 and G_2 !

Hierarchical Pooling In Practice

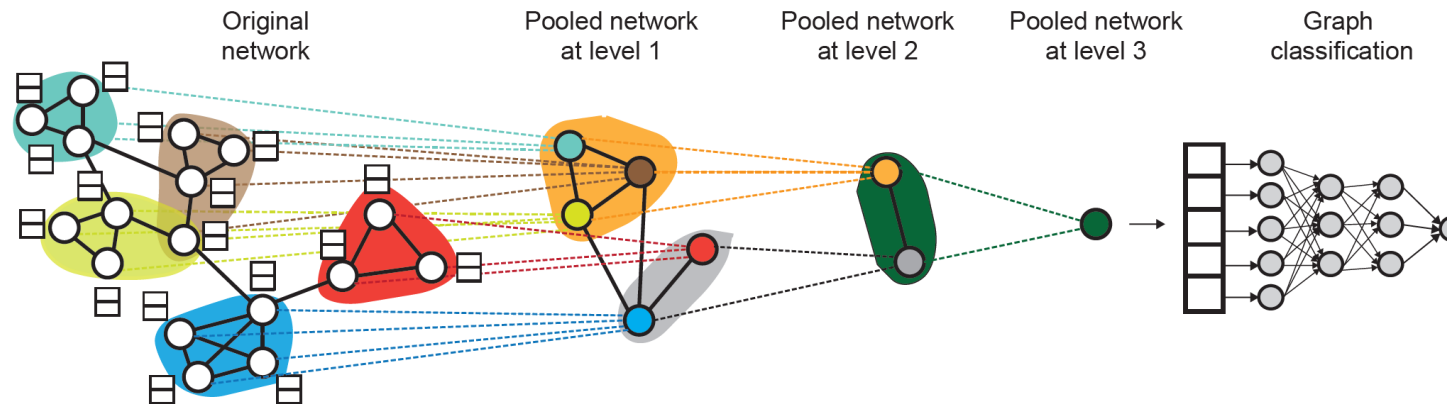
- **DiffPool idea:**
 - **Hierarchically pool node embeddings**



- **Leverage 2 independent GNNs at each level**
 - **GNN A:** Compute node embeddings
 - **GNN B:** Compute the cluster that a node belongs to
- **GNNs A and B at each level can be executed in parallel**

Hierarchical Pooling In Practice

- **DiffPool idea:**



- **For each Pooling layer**

- Use clustering assignments from **GNN B** to aggregate node embeddings generated by **GNN A**
- Create a **single new node** for each cluster, maintaining edges between clusters to generate a new **pooled** network

- **Jointly train GNN A and GNN B**

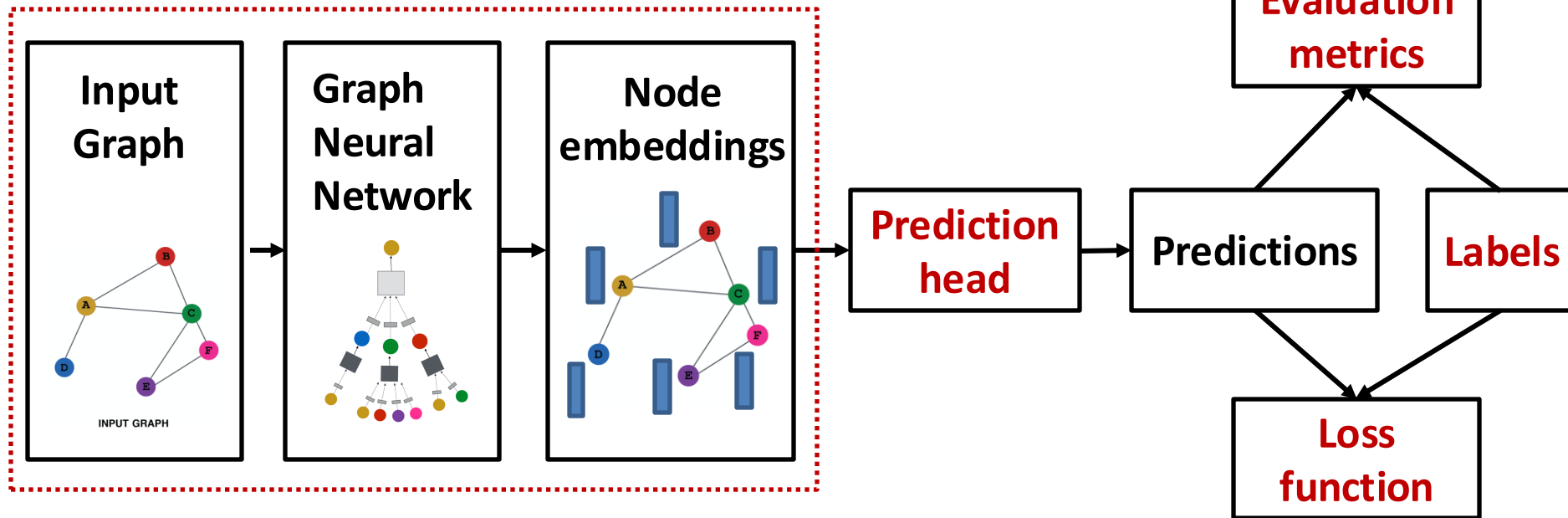
Graph Neural Networks: Model II

Training Graph Neural Networks

GNN Training Pipeline (2)

(2) Where does ground-truth come from?

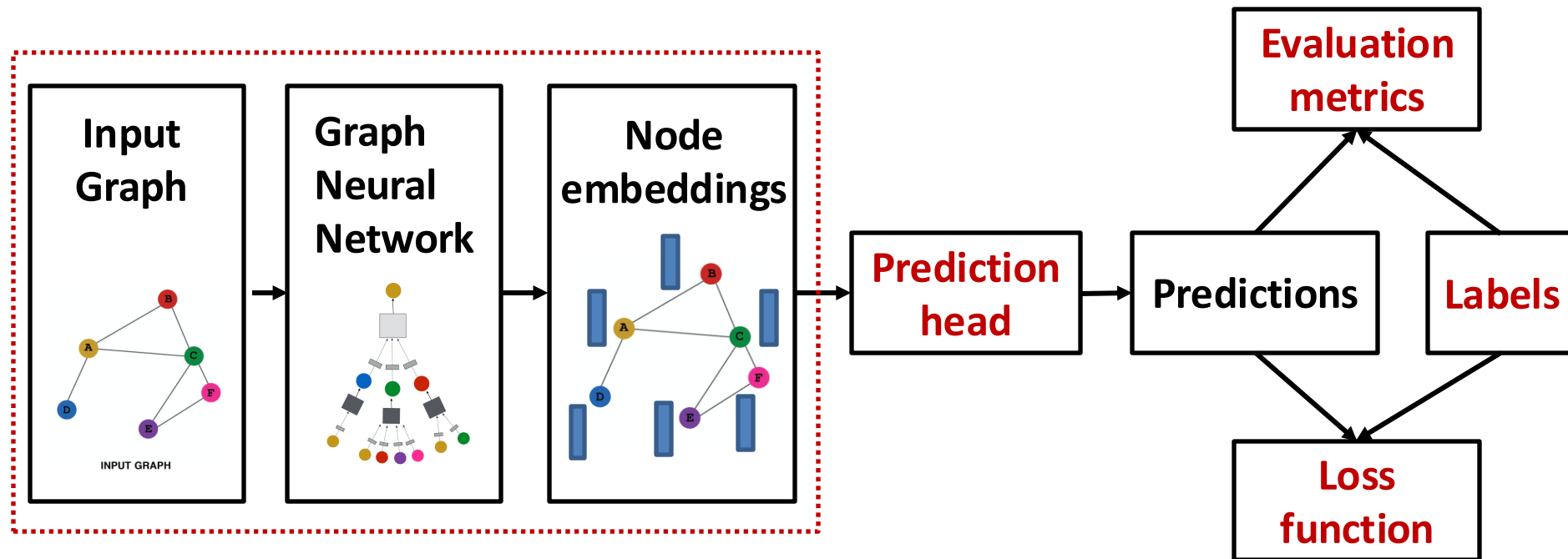
- Supervised labels
- Unsupervised signals



Recap: Supervised vs Unsupervised

- **Supervised learning on graphs**
 - **Labels come from external sources**
 - E.g., predict drug likeness of a molecular graph
- **Unsupervised learning on graphs**
 - **Signals come from graphs themselves**
 - E.g., link prediction: predict if two nodes are connected
- **Sometimes the differences are blurry**
 - We still have “supervision” in unsupervised learning
 - E.g., train a GNN to predict node clustering coefficient
 - An alternative name for “**unsupervised**” is “**self-supervised**”

GNN Training Pipeline (3)



(3) How do we compute the final loss?

- **Classification loss**
- **Regression loss**

Settings for GNN Training

- **The setting:** We have N data points
 - Each data point can be a node/edge/graph
 - **Node-level:** prediction $\hat{\mathbf{y}}_v^{(i)}$, label $\mathbf{y}_v^{(i)}$
 - **Edge-level:** prediction $\hat{\mathbf{y}}_{uv}^{(i)}$, label $\mathbf{y}_{uv}^{(i)}$
 - **Graph-level:** prediction $\hat{\mathbf{y}}_G^{(i)}$, label $\mathbf{y}_G^{(i)}$
 - We will use prediction $\hat{\mathbf{y}}^{(i)}$, label $\mathbf{y}^{(i)}$ to refer **predictions at all levels**

Classification or Regression

- **Classification:** labels $\mathbf{y}^{(i)}$ with discrete value
 - E.g., Node classification: which category does a node belong to
- **Regression:** labels $\mathbf{y}^{(i)}$ with continuous value
 - E.g., predict the drug likeness of a molecular graph
- GNNs can be applied to both settings
- **Differences: loss function & evaluation metrics**

Classification Loss

- As discussed in lecture 6, **cross entropy (CE)** is a very common loss function in classification
- K -way prediction** for i -th data point:

$$\text{CE}(\underbrace{\mathbf{y}^{(i)}}_{\text{Label}}, \underbrace{\hat{\mathbf{y}}^{(i)}}_{\text{Prediction}}) = - \sum_{j=1}^K \mathbf{y}_j^{(i)} \log(\hat{\mathbf{y}}_j^{(i)})$$

i -th data point
 j -th class

where:

E.g.

0	0	1	0	0
---	---	---	---	---

$\mathbf{y}^{(i)} \in \mathbb{R}^K$ = one-hot label encoding

$\hat{\mathbf{y}}^{(i)} \in \mathbb{R}^K$ = prediction after Softmax($\cdot$)

E.g.

0.1	0.3	0.4	0.1	0.1
-----	-----	-----	-----	-----

- Total loss over all N training examples

$$\text{Loss} = \sum_{i=1}^N \text{CE}(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)})$$

Regression Loss

- For regression tasks we often use **Mean Squared Error (MSE)** a.k.a. **L2 loss**
- K*-way regression for data point (i):

$$\text{MSE}(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)}) = \sum_{j=1}^K (\mathbf{y}_j^{(i)} - \hat{\mathbf{y}}_j^{(i)})^2$$

i-th data point
j-th target

where:

E.g.

1.4	2.3	1.0	0.5	0.6
-----	-----	-----	-----	-----

$\mathbf{y}^{(i)} \in \mathbb{R}^k$ = Real valued vector of targets

$\hat{\mathbf{y}}^{(i)} \in \mathbb{R}^k$ = Real valued vector of predictions

E.g.

0.9	2.8	2.0	0.3	0.8
-----	-----	-----	-----	-----

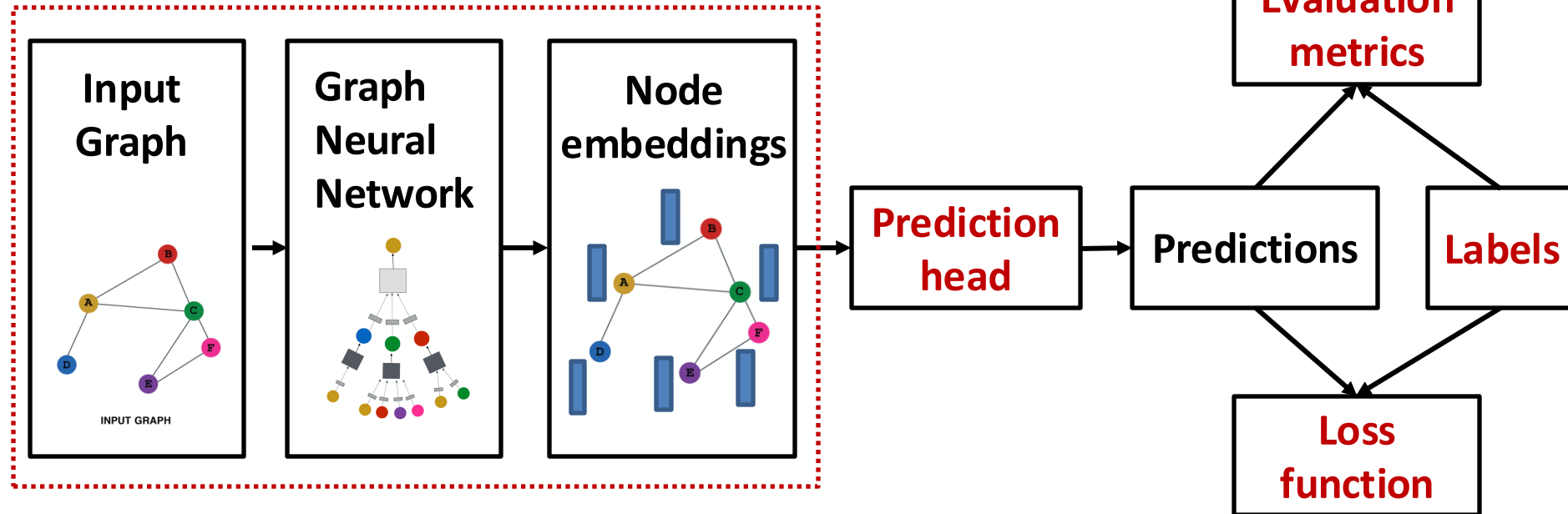
- Total loss over all *N* training examples

$$\text{Loss} = \sum_{i=1}^N \text{MSE}(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)})$$

GNN Training Pipeline (4)

(4) How do we measure the success of a GNN?

- Accuracy
- ROC AUC



Evaluation Metrics: Regression

- **We use standard evaluation metrics for GNN**
 - (Content below can be found in any ML course)
 - In practice we will use [sklearn](#) for implementation
 - Suppose we make predictions for N data points
- **Evaluate regression tasks on graphs:**
 - **Root mean square error (RMSE)**

$$\sqrt{\sum_{i=1}^N \frac{(\mathbf{y}^{(i)} - \hat{\mathbf{y}}^{(i)})^2}{N}}$$

- **Mean absolute error (MAE)**

$$\frac{\sum_{i=1}^N |\mathbf{y}^{(i)} - \hat{\mathbf{y}}^{(i)}|}{N}$$

Evaluation Metrics: Classification

- Evaluate classification tasks on graphs:

- (1) Multi-class classification

- We simply report the accuracy

$$\frac{1[\operatorname{argmax}(\hat{\mathbf{y}}^{(i)}) = \mathbf{y}^{(i)}]}{N}$$

- (2) Binary classification

- Metrics sensitive to classification threshold

- Accuracy

- Precision / Recall

- If the range of prediction is $[0,1]$, we will use 0.5 as threshold

- Metric Agnostic to classification threshold

- ROC AUC

Metrics for Binary Classification

- **Accuracy:**

$$\frac{TP + TN}{TP + TN + FP + FN} = \frac{TP + TN}{|\text{Dataset}|}$$

- **Precision (P):**

$$\frac{TP}{TP + FP}$$

- **Recall (R):**

$$\frac{TP}{TP + FN}$$

- **F1-Score:**

$$\frac{2P * R}{P + R}$$

Confusion matrix

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Evaluation Metrics

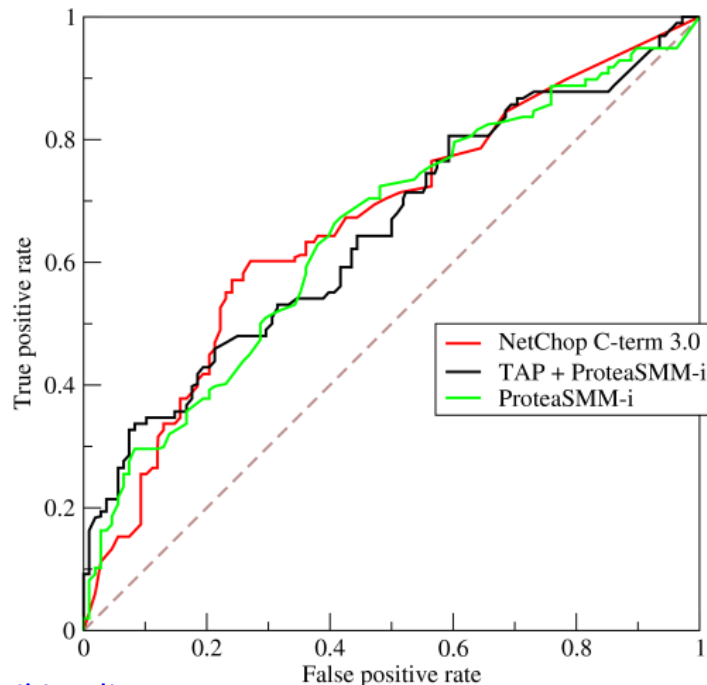
- **ROC Curve:** Captures the tradeoff in TPR and FPR **as the classification threshold is varied for a binary classifier.**

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

TPR

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

Image Credit: [Wikipedia](#)



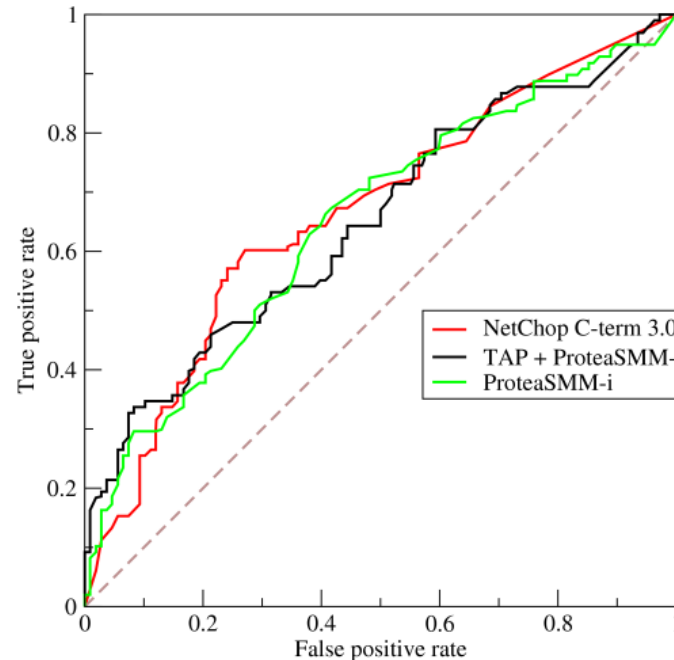
FPR

Note: the dashed line represents performance of a random classifier

ROC AUC: Area under the ROC Curve

- **Intuition:** The probability that a classifier will rank a randomly chosen **positive** instance **higher** than a randomly chosen negative one

(4) Evaluation Metrics



Content Credit: [Wikipedia](#)

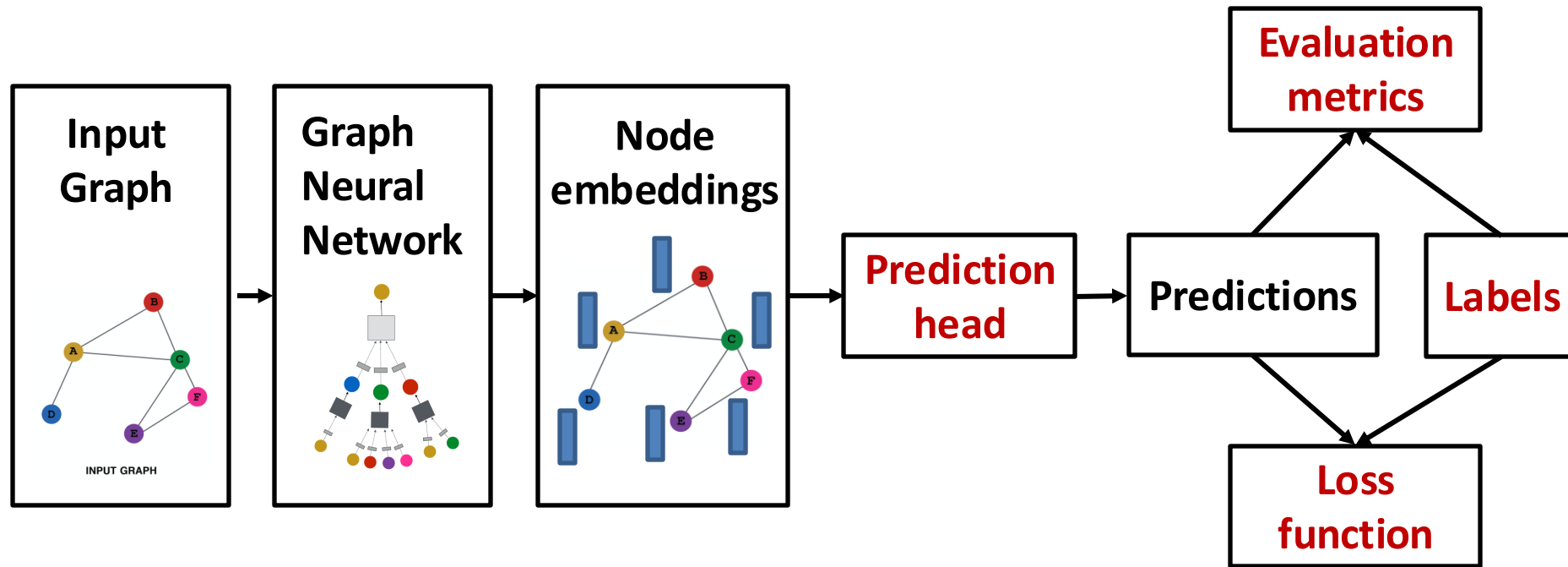
ROC AUC: Area under the ROC Curve.

- **Intuition:** The probability that a classifier will rank a randomly chosen positive instance higher than a randomly chosen negative one

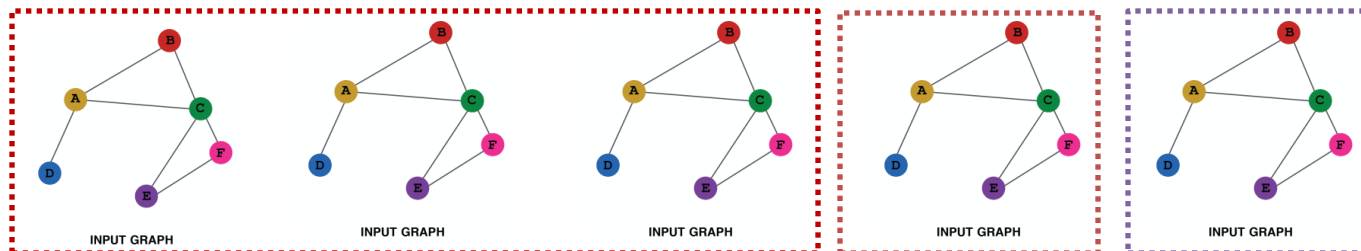
Graph Neural Networks: Model II

Setting-up GNN Prediction Tasks

GNN Training Pipeline (5)



(5) How do we split our dataset into train / validation / test set?



Dataset split

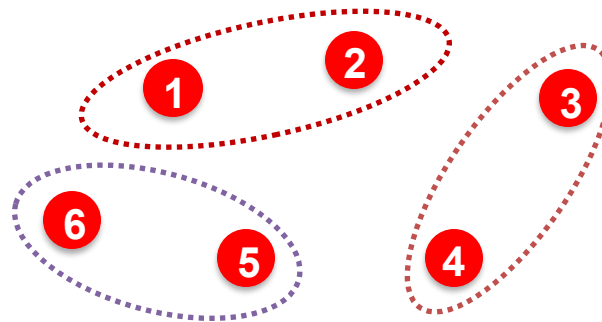
Dataset Split: Fixed / Random Split

- **Fixed split:** We will split our dataset **once**
 - **Training set:** used for optimizing GNN parameters
 - **Validation set:** develop model/hyperparameters
 - **Test set:** held out until we report final performance
- **A concern:** sometimes we cannot guarantee that the test set will really be held out
- **Random split:** we will **randomly split** our dataset into training / validation / test
 - We report **average performance over different random seeds**

Why Splitting Graphs is Special

- Suppose we want to split an image dataset
 - **Image classification:** Each data point is an image
 - Here **data points are independent**
 - Image 5 will not affect our prediction on image 1

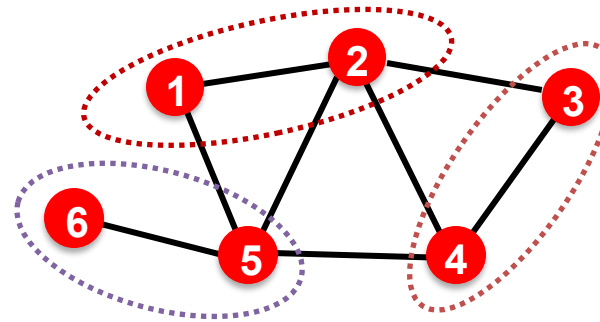
Training
Validation
Test



Why Splitting Graphs is Special

- **Splitting a graph dataset is different!**
 - **Node classification:** Each data point is a node
 - Here **data points are NOT independent**
 - **Node 5 will affect our prediction on node 1**, because it will participate in message passing → affect node 1's embedding

Training
Validation
Test



- **What are our options?**

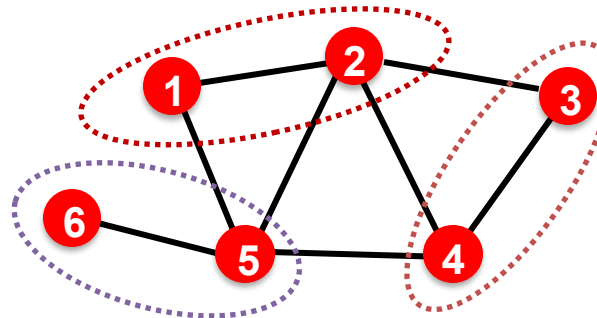
Why Splitting Graphs is Special

- **Solution 1 (Transductive setting):** The input graph can be observed in all the dataset splits (training, validation and test set).
- We will only split the (node) labels
 - At training time, we compute embeddings using the entire graph, and train using node 1&2's labels
 - At validation time, we compute embeddings using the entire graph, and evaluate on node 3&4's labels

Training

Validation

Test



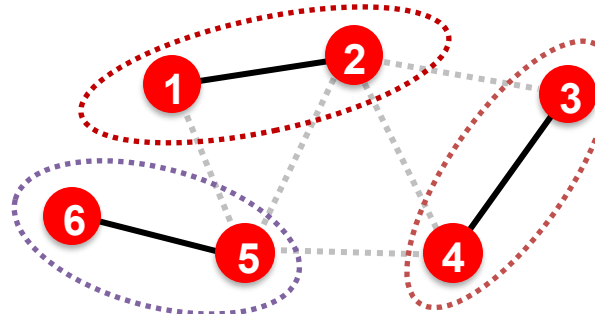
Why Splitting Graphs is Special

- **Solution 2 (Inductive setting):** We break the edges between splits **to get multiple graphs**
 - **Now we have 3 graphs that are independent.** Node 5 will not affect our prediction on node 1 any more
 - **At training time**, we compute embeddings **using the graph over node 1&2**, and train **using node 1&2's labels**
 - **At validation time**, we compute embeddings **using the graph over node 1&2&3&4**, and **evaluate on node 3&4's labels**

Training

Validation

Test



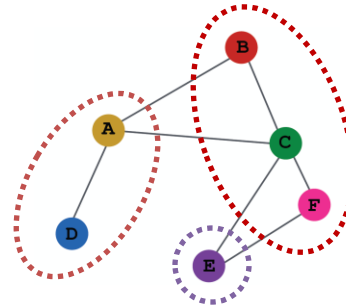
Transductive / Inductive Settings

- **Transductive setting:** training / validation / test sets observe **the same graph**, only **labels are held out**
 - The **dataset usually consists of one graph**
 - Only applicable to **node / edge** prediction tasks
- **Inductive setting:** training / validation / test sets observe different parts of the dataset, both **data and labels are held out**
 - The **dataset usually consists of multiple graphs (or subgraphs)**
 - A successful model should **generalize to unseen graphs (subgraph)**
 - Applicable to **node / edge / graph** tasks

Example: Node Classification

- **Transductive** node classification

- All the splits can observe the entire graph structure, but can only observe the labels of their respective nodes



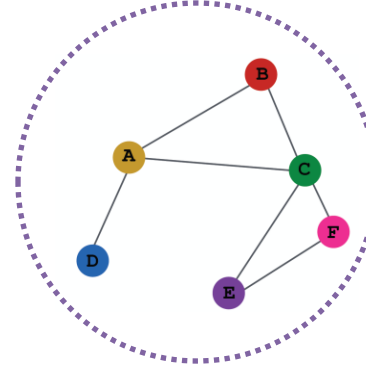
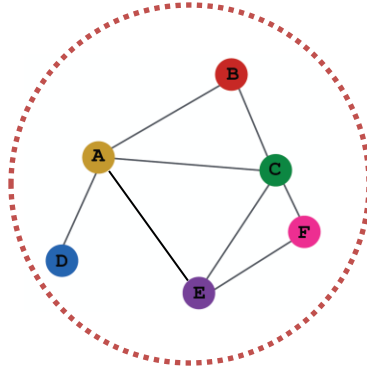
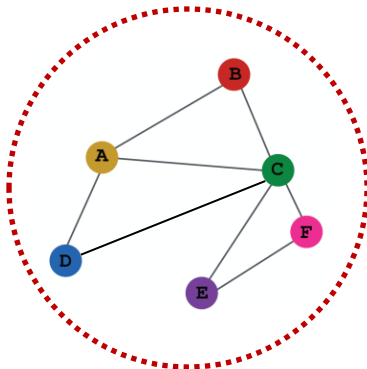
Training

Validation

Test

- **Inductive** node classification

- Suppose we have a dataset of 3 graphs
- Each split contains an independent graph



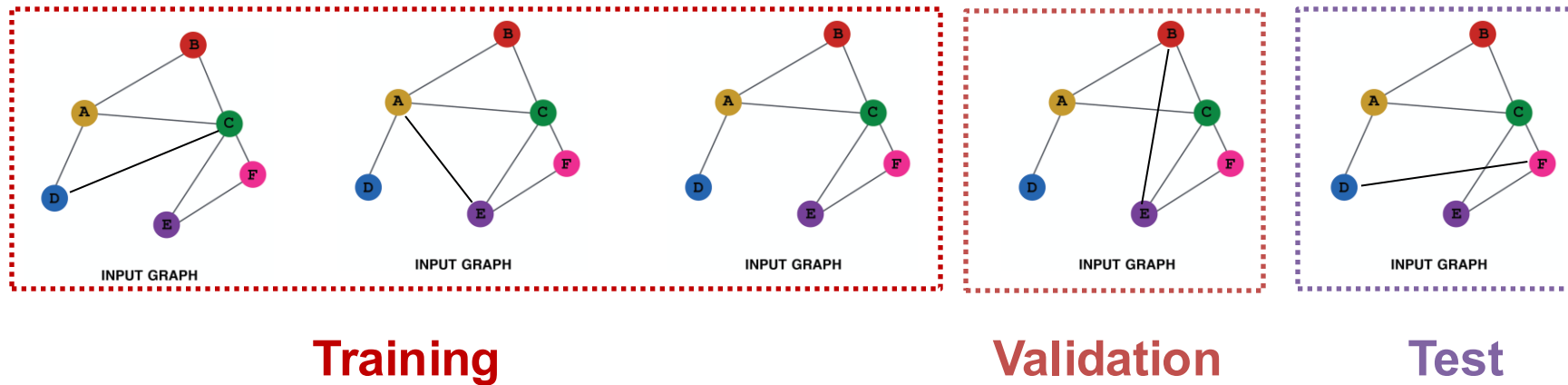
Training

Validation

Test

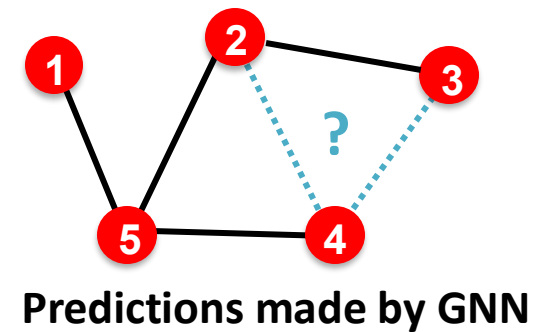
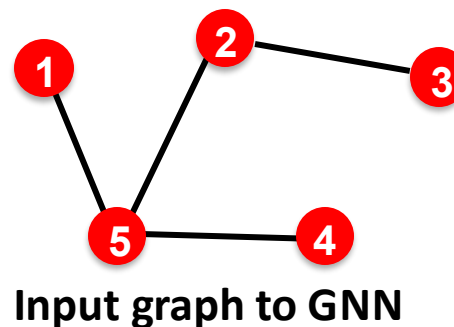
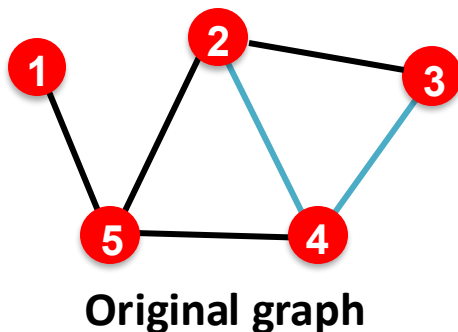
Example: Graph Classification

- Only the **inductive setting** is well defined for **graph classification**
 - Because **we have to test on unseen graphs**
 - Suppose we have a dataset of 5 graphs. Each split will contain independent graph(s).

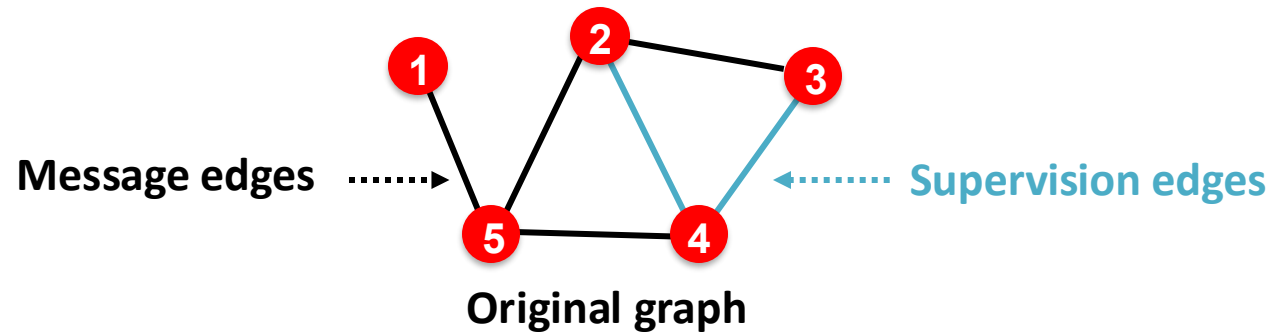


Example: Link Prediction

- **Goal of link prediction:** predict missing edges
- **Setting up link prediction is tricky:**
 - Link prediction is an unsupervised / self-supervised task. We need to **create the labels** and **dataset splits** on our own
 - Concretely, we need to **hide some edges from the GNN** and **let the GNN predict if the edges exist**
 - Important for Knowledge graph completion & reasoning as well!



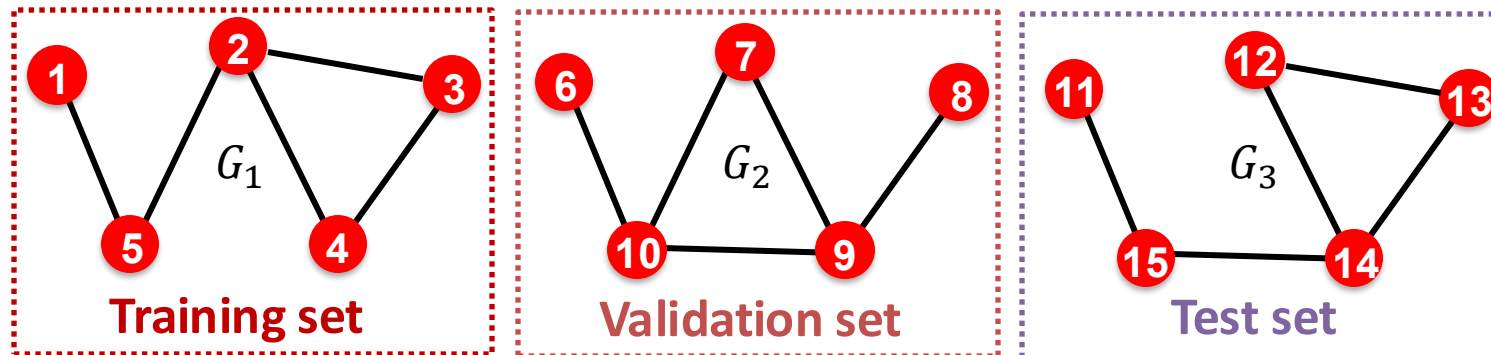
Setting up Link Prediction



- For link prediction, we will split edges twice
- Step 1: **Assign 2 types of edges in the original graph**
 - Message edges: Used for GNN message passing
 - Supervision edges: Use for computing objectives
 - After step 1:
 - Only message edges will remain in the graph
 - Supervision edges are used as supervision for edge predictions made by the model, will not be fed into GNN!

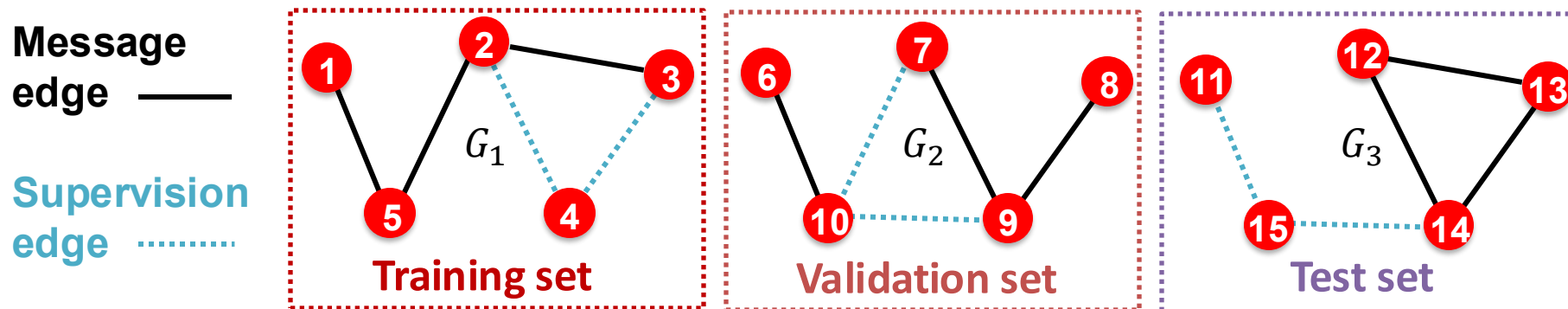
Setting up Link Prediction

- **Step 2: Split edges into train / validation / test**
- **Option 1: Inductive link prediction split**
 - Suppose we have a dataset of 3 graphs. Each inductive split will contain an independent graph



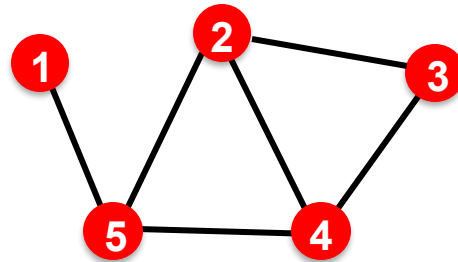
Setting up Link Prediction

- Step 2: **Split edges into train / validation / test**
- Option 1: Inductive link prediction split
 - Suppose we have a dataset of 3 graphs. Each inductive split will contain an independent graph
 - In **train** or **val** or **test** set, each graph will have **2** types of edges: message edges + supervision edges
 - Supervision edges are not the input to GNN



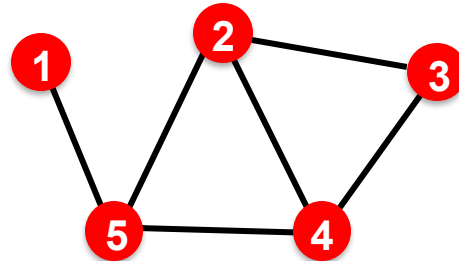
Setting up Link Prediction

- **Option 2: Transductive link prediction split:**
 - **This is the default setting when people talk about link prediction**
 - **Suppose we have a dataset of 1 graph**

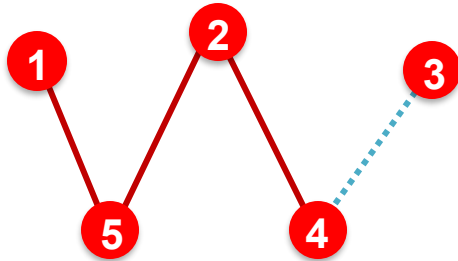


Setting up Link Prediction

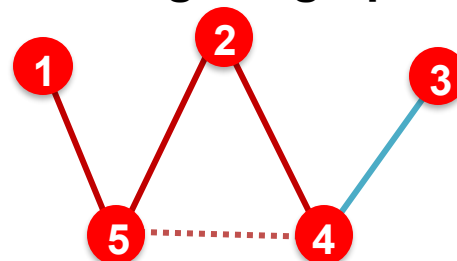
- Option 2: Transductive link prediction split:



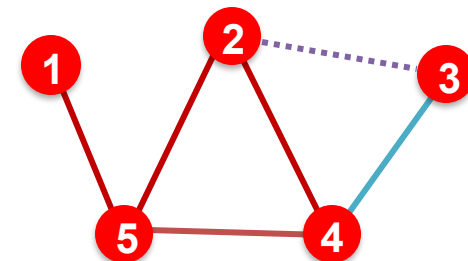
The original graph



(1) At training time:
Use **training message edges** to predict **training supervision edges**



(2) At validation time:
Use **training message edges** & **training supervision edges** to predict **validation edges**



(3) At test time:
Use **training message edges** & **training supervision edges** & **validation edges** to predict **test edges**

Setting up Link Prediction

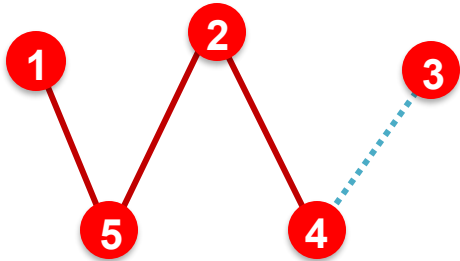
■ Option 2: Transductive link prediction split:

Why do we use growing number of edges?

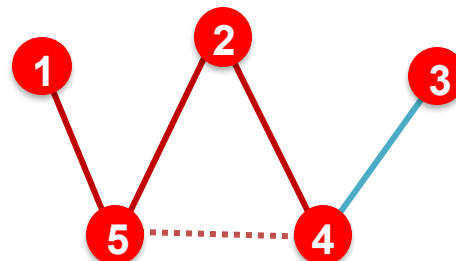
After training, supervision edges are known to GNN.

Therefore, an ideal model should use supervision edges in message passing at validation time.

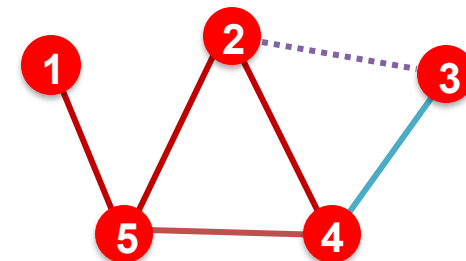
The same applies to the test time.



(1) At training time:
Use **training message edges** to predict **training supervision edges**



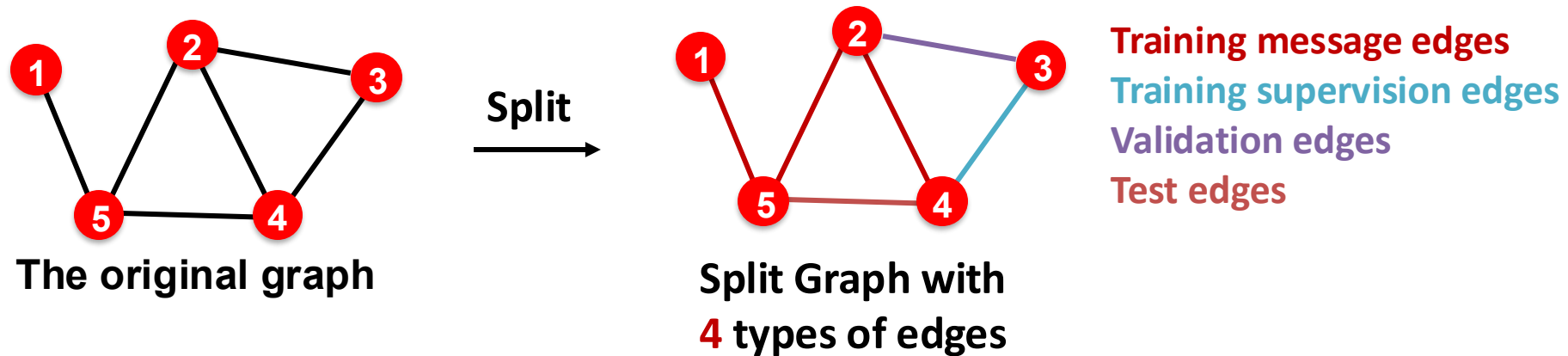
(2) At validation time:
Use **training message edges & training supervision edges** to predict **validation edges**



(3) At test time:
Use **training message edges & training supervision edges & validation edges** to predict **test edges**

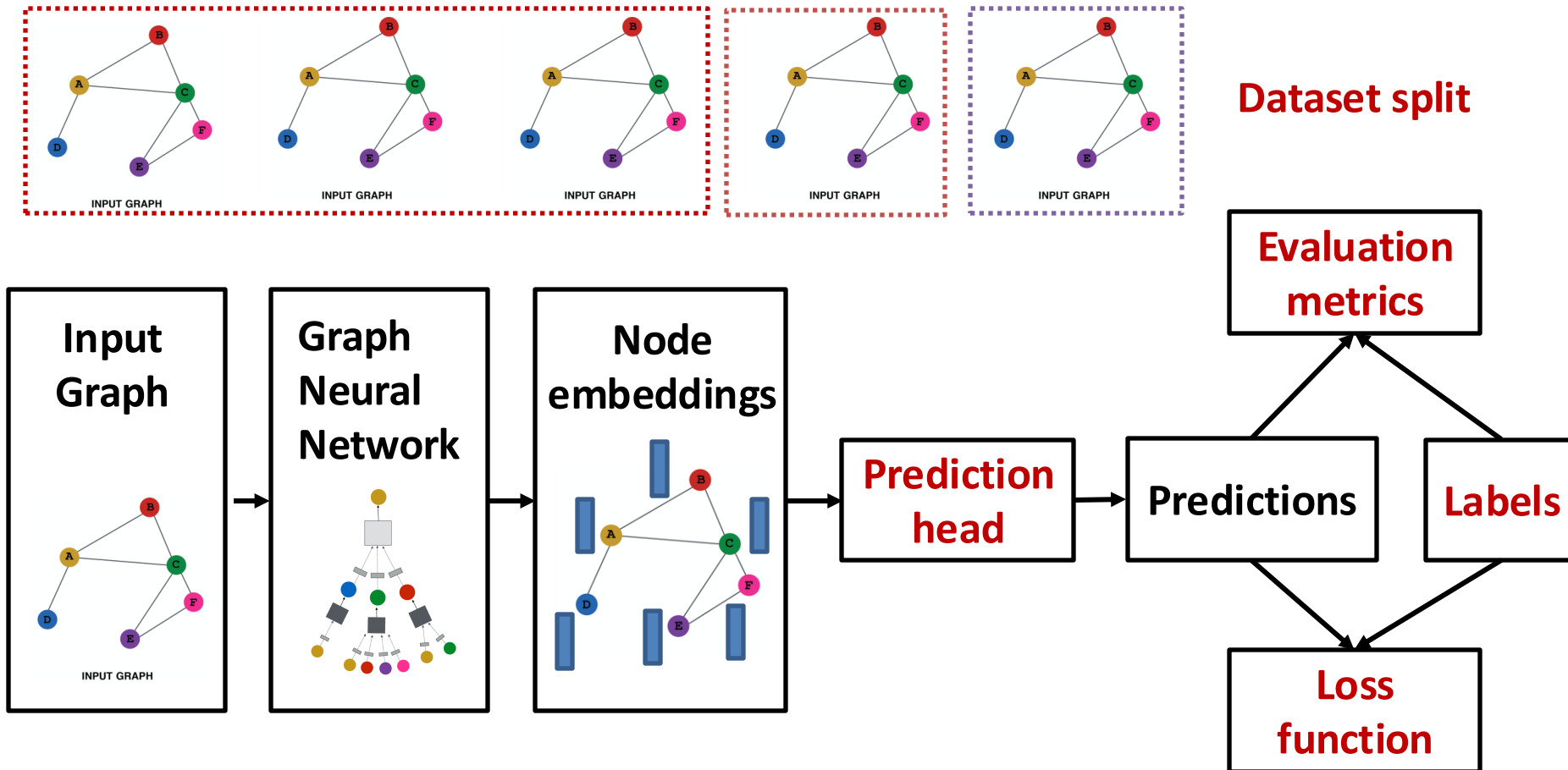
Setting up Link Prediction

- **Summary: Transductive link prediction split:**



- **Note:** Link prediction settings are tricky and complex. You may find papers do link prediction differently.
- Luckily, we have full support in **PyG** and [GraphGym](#)

GNN Training Pipeline



Implementation resources:

PyG provides core modules for this pipeline

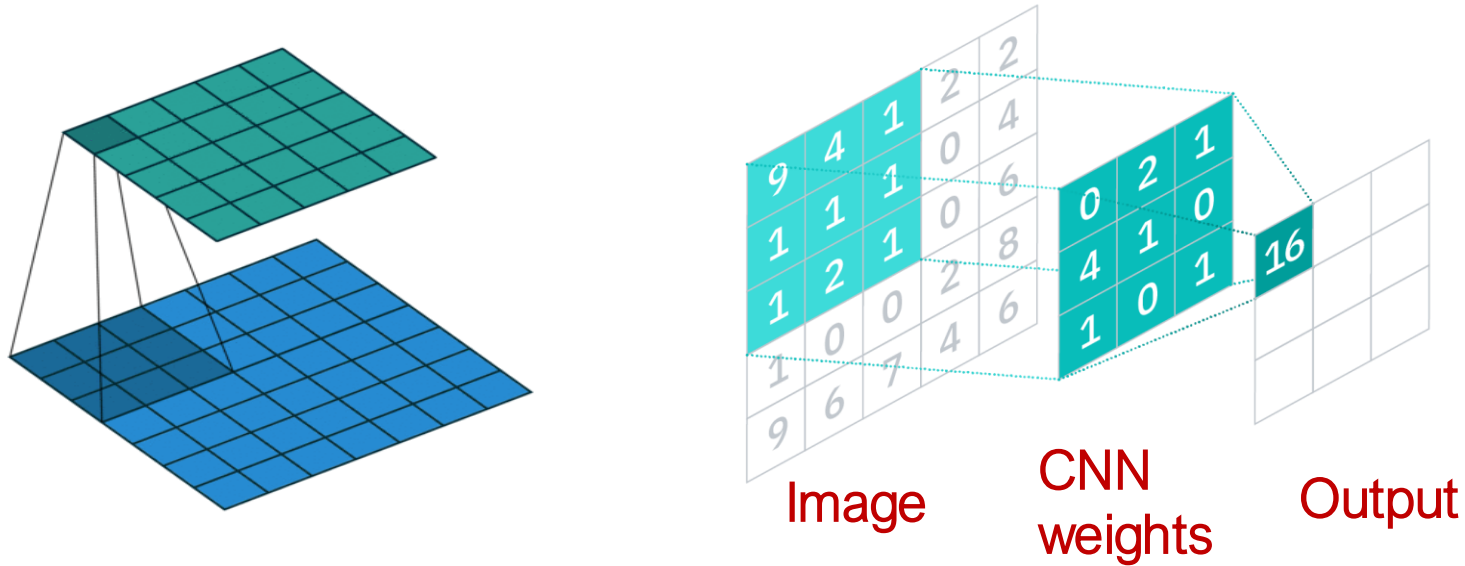
GraphGym further implements the full pipeline to facilitate GNN design

Graph Neural Networks: Perspective

GNNs vs CNNs and Transformers

Convolutional Neural Network

Convolutional neural network (CNN) layer with 3x3 filter:



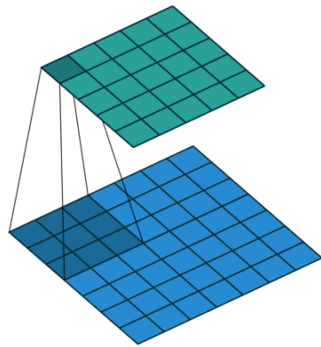
CNN formulation: $h_v^{(l+1)} = \sigma(\sum_{u \in N(v) \cup \{v\}} W_l^u h_u^{(l)}), \quad \forall l \in \{0, \dots, L-1\}$

if we rewrite: $h_v^{(l+1)} = \sigma(\sum_{u \in N(v)} \mathbf{W}_l^u h_u^{(l)} + B_l h_v^{(l)}), \quad \forall l \in \{0, \dots, L-1\}$

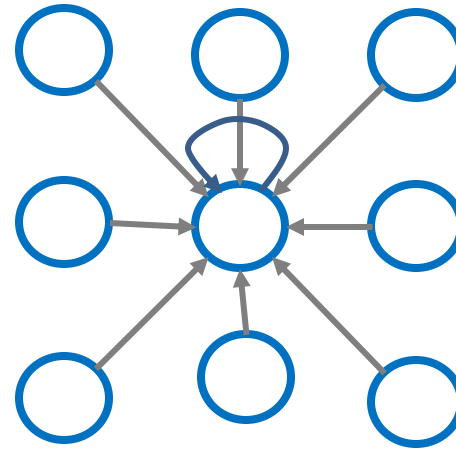
$N(v)$ represents the 8 neighbor pixels of v .

GNN vs. CNN

Convolutional neural network (CNN) layer with 3x3 filter:



Image

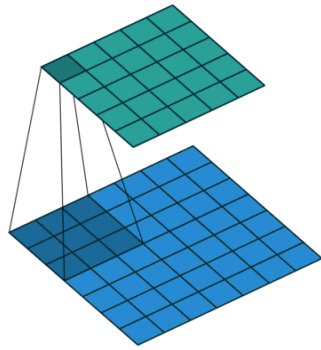


Graph

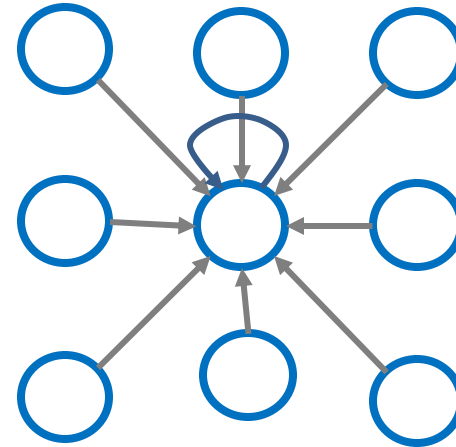
- GNN formulation: $h_v^{(l+1)} = \sigma(\mathbf{W}_l \sum_{u \in N(v)} \frac{h_u^{(l)}}{|N(v)|} + B_l h_v^{(l)}), \forall l \in \{0, \dots, L-1\}$
- CNN formulation: $h_v^{(l+1)} = \sigma(\sum_{u \in N(v)} \mathbf{W}_l^u h_u^{(l)} + B_l h_v^{(l)}), \forall l \in \{0, \dots, L-1\}$

GNN vs. CNN

Convolutional neural network (CNN) layer with 3x3 filter:



Image



Graph

$$\text{GNN formulation: } h_v^{(l+1)} = \sigma(\mathbf{W}_l \sum_{u \in N(v)} \frac{h_u^{(l)}}{|N(v)|} + B_l h_v^{(l)}), \forall l \in \{0, \dots, L-1\}$$

$$\text{CNN formulation: } h_v^{(l+1)} = \sigma(\sum_{u \in N(v)} \mathbf{W}_l^u h_u^{(l)} + B_l h_v^{(l)}), \forall l \in \{0, \dots, L-1\}$$

Key difference: We can learn different $\mathbf{W}_l^u$ for different “neighbor” u for pixel v on the image. The reason is we can define a canonical order for the 9 neighbors using **relative position** to the center pixel: $\{(-1,-1), (-1,0), (-1,1), \dots, (1,1)\}$ (in other words, image is an ordered data structure)

GNN vs. CNN

Convolutional neural network (CNN) layer with 3x3

CNN can be seen as a GNN with fixed neighbor size and non permutation invariant message function:

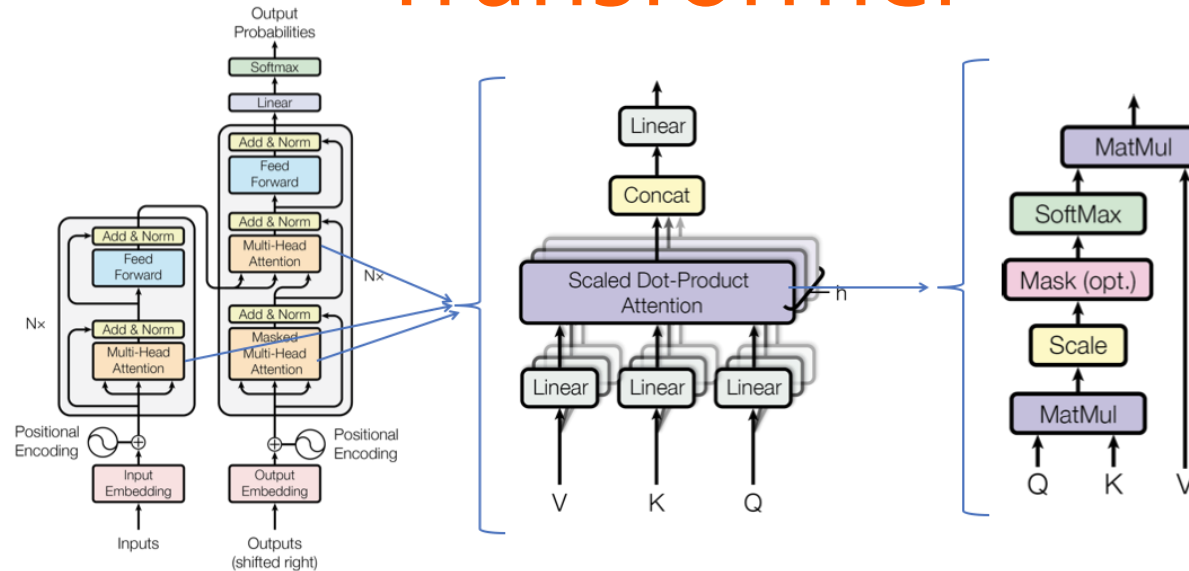
- The size of the filter is pre-defined for a CNN.
- The advantage of GNN is it processes **arbitrary graphs** with different degrees for each node.
- The advantage of CNN is that it's **more expressive** since it can leverage the canonical order in an image

CNN is not permutation invariant/equivariant.

- Switching the order of pixels will leads to different outputs.

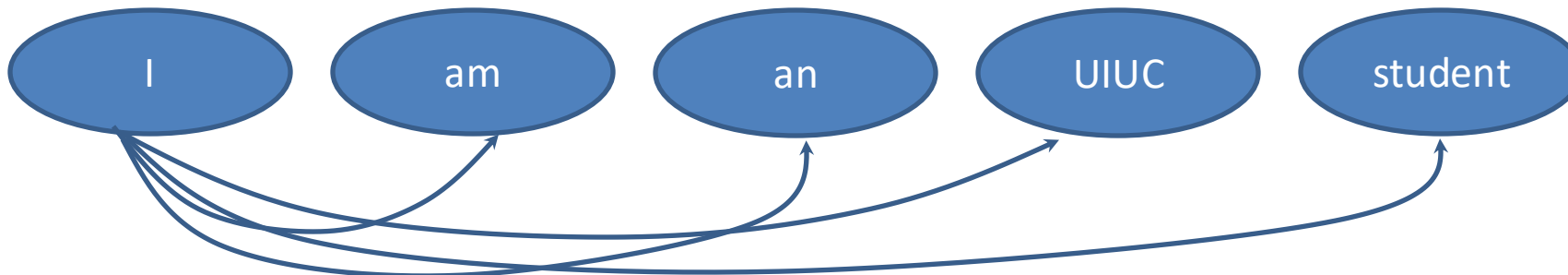
Key difference: We can learn different W_l^u for different “neighbor” u for pixel v on the image. The reason is we can define a canonical order for the 9 neighbors using **relative position** to the center pixel: $\{(-1,-1), (-1,0), (-1,1), \dots, (1,1)\}$ (in other words, image is an ordered data structure)

Transformer



Key component: self-attention

- Every token/word attends to all the other tokens/words via matrix calculation.



Recall: GAT is a Sparse Transformer

- **Normalize** e_{vu} into the **final attention weight** α_{vu}
 - Use the **softmax** function, so that $\sum_{u \in N(v)} \alpha_{vu} = 1$:

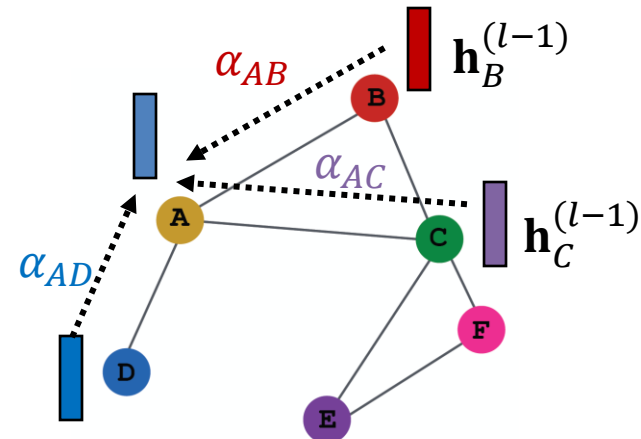
$$\alpha_{vu} = \frac{\exp(e_{vu})}{\sum_{k \in N(v)} \exp(e_{vk})}$$

- **Weighted sum** based on the **final attention weight** α_{vu}

$$\mathbf{h}_v^{(l)} = \sigma\left(\sum_{u \in N(v)} \alpha_{vu} \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}\right)$$

Weighted sum using α_{AB} , α_{AC} , α_{AD} :

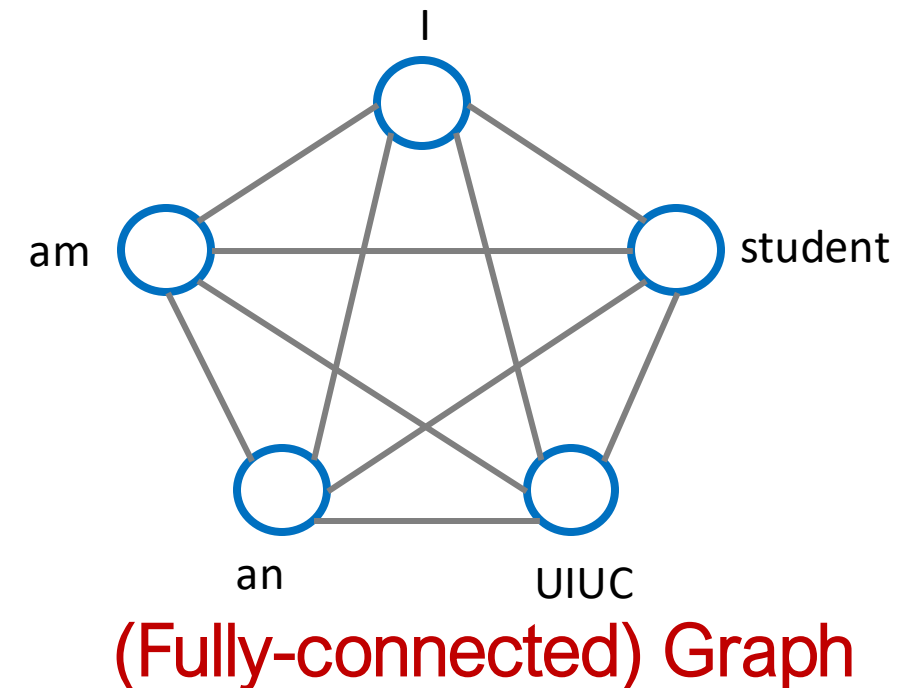
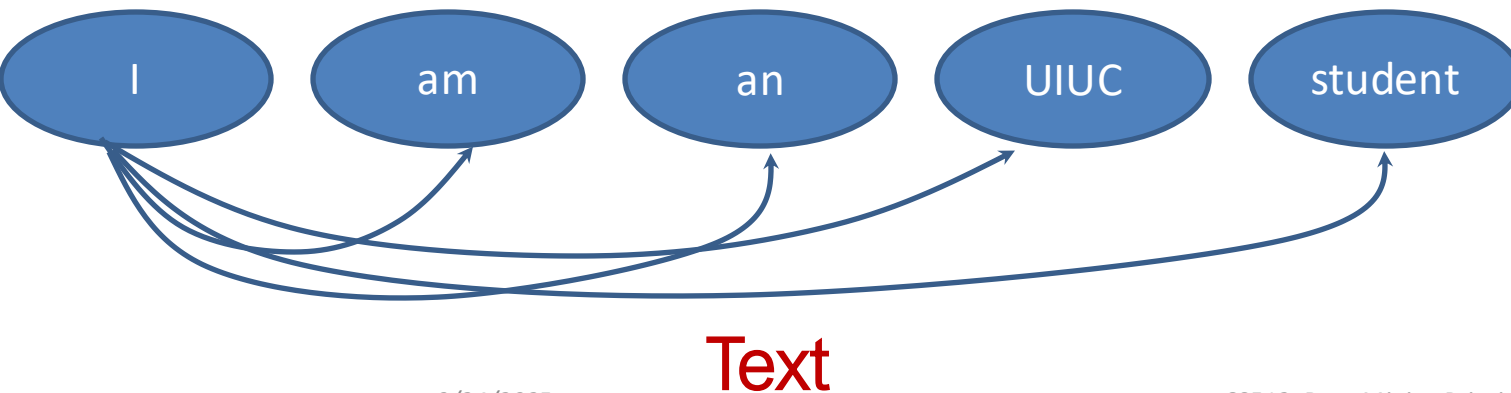
$$\mathbf{h}_A^{(l)} = \sigma\left(\alpha_{AB} \mathbf{W}^{(l)} \mathbf{h}_B^{(l-1)} + \alpha_{AC} \mathbf{W}^{(l)} \mathbf{h}_C^{(l-1)} + \alpha_{AD} \mathbf{W}^{(l)} \mathbf{h}_D^{(l-1)}\right)$$



GNN vs. Transformer

Transformer layer can be seen as a special GNN that runs on a fully-connected “word” graph!

Since each word attends to **all the other words**, **the computation graph** of a transformer layer is identical to that of a GNN on the **fully-connected “word” graph**.



GNN vs. Transformer

	GNN	Transformer
Model architecture	A large & flexible model design space, including GAT that mimics Transformer	A specific NN architecture (minor changes have been made over the years)
Input data – order info	Permutation invariant, no positional encoding	Sequence order -> positional encoding (abs encoding added in the 1 st layer, relative encoding added every layer)
Input data – relational info	Encoded as sparsity pattern in attention	No relational info -> fully connected graph
Output	Node embeddings + diverse node/edge/graph prediction heads	Node/Token embeddings. Could have “node/edge/graph” heads, but mostly node prediction heads

Summary of the Lecture

- We introduce **a general GNN framework**:
 - **GNN Layer**:
 - Transformation + Aggregation. Classic GNN layers: GCN, GraphSAGE, GAT
 - **Layer connectivity**:
 - The over-smoothing problem. Solution: skip connections
 - **Graph Augmentation**:
 - Feature augmentation. Structure augmentation
 - **Learning Objectives**
 - The full training pipeline of a GNN
- And **compared GNN to other common NNs**: CNNs & Transformer